Peer Feedback App

by

Ethan Cox, Advait Parab, Melanie Peck, Nathaniel Wisher

Submitted to
the Faculty of the School of Information Technology
in Partial Fulfillment of the Requirements for
the Degree of Bachelor of Science
in Information Technology AND/OR Cybersecurity

© Copyright 2025 STUDENT NAMES

The author grants to the School of Information Technology permission
to reproduce and distribute copies of this document in whole or in part.

Ethan Cox	11/02/2025
STUDENT 1	Date
Advait Parab	11/02/2025
STUDENT 2	Date
Melanie Peck	11/02/2025
STUDENT 3	Date
Nathaniel Wisher	11/02/2025
STUDENT 4	Date

_____ _____

ADVISOR , Faculty Advisor Date

University of Cincinnati
College of
Education, Criminal Justice, and Human Services

TABLE OF CONTENTS

Contents

TABLE OF CONTENTS 3
LIST OF FIGURES 5
ABSTRACT 6
INTRODUCTION 7
Introduction 7
Problem 7
Solution 7
Project Goals 8
Overview 8
DISCUSSION 10
Project Concept 10
Design Objectives 10
Methodology and Technical Approach 11
User Profiles/Personas 13
Student Persona - Jordan Ruiz 13
Instructor Persona - Dr. Priya Menon 13
Administrator Persona - Marcus Hale 14
Use Case Diagrams 14
Actors and Major Use Cases: 14
Use Case Diagram Description (Expanded Narrative) 15
Technical Architecture 17
Testing 20
Testing Objectives 20
Testing Methods 20
Testing Results and Findings 21
Budget 22
Budget Discussion 23
Project Plan 24
Project Plan Discussion 27
Problems Encountered and Analysis of Problems Solved 27

1. Responsive Design and Layout Inconsistencies 27
2. Time Constraints and Task Prioritization 28
3. Limited Backend Connectivity 28
Summary 29
Recommendations for Improvement 29
1. Implement Full Backend Integration Early 29
2. Introduce Continuous Integration and Deployment (CI/CD) 29
3. Strengthen Testing Coverage 30
4. Enhance Accessibility and Usability 30
5. Improve Documentation and Knowledge Sharing 30
Summary 30
CONCLUSION 31
Lessons Learned 31
Abilities and Skills Developed Throughout Project 32
Technical Skills Developed 32
Professional and Interpersonal Skills Developed 33
Plans For Spring 34
REFERENCES AND APPENDIX 36
References 36
Appendix 37
Appendix A – Frontend Interface Mockups 37
Appendix B – Technologies and Tools 38

LIST OF FIGURES

Figure 1. Use Case Diagram 16
Figure 2. Logical Network Diagram 19
Figure 3. Project Budget Summary 23
Figure 4. Project Timeline 26
Figure 5. Login Page 37
Figure 6. Feedback Submission Form 38
Figure 7. Student Dashboard 38
Figure 8. Table of Technologies and Tools 39

ABSTRACT

Commented [EC1]: Add Content Here

The Peer Feedback App is a web-based platform developed to facilitate structured, anonymous, and constructive peer evaluations among students. Recognizing common barriers such as stigma, cost, and accessibility that hinder students from seeking feedback and help, this app provides a low-cost, secure, and accessible environment for honest peer-to-peer feedback. The system allows students to submit anonymous assessments based on criteria such as standard or custom ones, supports self-evaluation and growth tracking, and provides feedback dashboards for effective performance evaluation. This application was developed using Django for backend services and ReactJS for the frontend interface. Data privacy is ensured using strong measures that are compliant with FERPA and GDPR. This effort aims at improving collaboration and academic success in student projects by fostering accountability and continuous improvement. It used agile development methodologies where iterative design and testing, along with user feedback, were emphasized to ensure usability and reliability. The following report describes the problem context, technical architecture, development process, testing strategies, and future recommendations for developing the capabilities of the app. At this stage, only the frontend has been completed, with backend integration planned for Spring 2026.

INTRODUCTION

Introduction

Peer Feedback App aims to enhance student work and collaboration through simplified participation in structured, anonymous, and constructive peer feedback. Students are usually faced with problems such as stigma, cost, and accessibility barriers. A study conducted on 345 struggling university students identified stigma against tutoring as the primary inhibitor of seeking tutoring (Ciscell et al., 2016). Embracing peer review as a beneficial way to facilitate learning and development (Simonsmeier et al., 2020), the app provides a secure online platform in which students can offer and receive anonymous feedback on various measures of performance, maintaining accountability and continuous development.

Problem

Students do not want to seek assistance since they fear being criticized or bear the expense and effort of hiring a tutor. Traditional peer feedback mechanisms lack structure and anonymity and hence encounter low participation and less candid feedback. Inadequate timely, significant, and constructive feedback impacts learning performance as well as skills development. This project addresses these challenges by creating an application that will allow constructive, peer review anonymously in a comfortable, supportive environment.

Solution

Our service is a peer review app through which students can evaluate each other on pre-determined or user-specified criteria anonymously. The app enables:

- A structured anonymous feedback mechanism encouraging honest critique.
- Self-evaluation systems with growth tracking to help users review over time.
- Feedback dashboards with received reviews aggregated.

- Reminders and notifications to encourage timely engagement.
- Strict data privacy and security regulations compliance (FERPA, GDPR).

The app is centered on accessibility, privacy, and simplicity and creates a culture of one-up-manship where students exchange and receive feedback from fellow students. The practice also enhances professional communication skills through learning how to give good feedback (Erasmus University Rotterdam, n.d.).

Project Goals

The overall objective is to develop an intuitive, user-friendly application to facilitate structured, anonymous peer feedback for students.

Objectives include:

- Enable anonymous peer feedback on either standard or customized criteria.
- Enable self-assessment and reflection to track progress and development.
- Develop a positive and respectful culture of peer feedback through guidelines for feedback.
- Prioritize data privacy and enable easy access via an intuitive interface.

Overview

Commented [EC2]: Add Content Here

The Peer Feedback App is meant to fill in the gap by providing a structured, anonymous platform for peer-to-peer feedback among students. Traditional methods of obtaining feedback often lack anonymity and structure. These barriers negatively impact honest communication and limit opportunities for growth. This application enables students to give and receive constructive evaluations based on customizable criteria, supporting academic and personal development.

The key principles for the app's design involve simplicity, accessibility, and privacy. It features secure user registration and authentication using university credentials, anonymous submission of peer evaluations, real-time feedback dashboards, and automated notifications to

promote timely participation. By integrating self-assessment tools alongside peer feedback, the app encourages reflection and accountability in its users.

The system technically uses a ReactJS frontend to provide an interactive, responsive interface, while the Django backend handles the business logic and secure data processing. Data is stored reliably in a relational SQL Server database, and OAuth 2.0 secures authentication and access.

During development, the project followed agile methodologies of iterative design, implementation, and testing while incorporating user feedback in refining functionality and usability. Care has been taken to define the scope of the project to implement critical features within this semester, with further enhancement planned for mobile support, media attachments, and more advanced analytics.

It contains a description of the problem analysis, proposed solution, technical architecture of the system, development and testing phases, team roles, ethical considerations, and recommendations for future work. This application will help to provide an effective and supportive educational environment since it encourages honest feedback, increases the autonomy of students, and helps students enhance the collaborative skills that are much needed in academic and professional life.

DISCUSSION

Project Concept

The app bypasses student barriers of feedback anxiety and limited access to tutoring through the provision of:

- University credentials or single sign-on to user registration and authentication.
- Anonymous peer feedback on fixed or custom criteria with feedback prompts.
- Aggregated feedback dashboards for open-ended performance summaries.
- Automated reminders for on-time submission of feedback.
- Comprehensive privacy protections such as encryption and compliance with FERPA and GDPR standards.

Features outside the scope of this release include peer matching, specialized mobile apps, media attachment support, exportable reports, cloud archiving, cross-platform notifications, and offline capabilities, to be added later.

Design Objectives

Commented [EC3]: Add content here

The design of the Peer Feedback App is guided by several core objectives intended to promote constructive communication, preserve privacy, and ensure long-term scalability. A primary objective is to protect the anonymity of students throughout the feedback process. To achieve this, the system will separate identifying information from submitted feedback at the database level, preventing instructors, administrators, or system maintainers from linking a specific comment to a student. This approach supports compliance with FERPA while encouraging students to provide honest, reflective responses without fear of social consequences.

Another major objective is to maintain strong data security and regulatory compliance. The application will leverage OAuth 2.0 for authentication, enforce role-based access controls, and store all data in SQL Server using secure, encrypted connections. The design also emphasizes structured feedback prompts that help students provide meaningful, criteria-focused evaluations that align with course learning outcomes. This encourages consistency and reliability across peer assessments.

Usability and accessibility also play a central role in the system's design. The interface is intentionally simple, featuring clean layouts, clear navigation, and responsive components that work effectively on both desktop and mobile devices. This ensures that students and instructors can quickly understand and use the platform with minimal instruction. In addition, the system is designed with extensibility in mind. Future enhancements, such as advanced analytics dashboards, multimedia attachments, and improved instructor-level controls, can be added without disrupting the underlying architecture.

Collectively, these design objectives establish a foundation for a secure, transparent, and educationally meaningful feedback platform that supports both immediate course needs and long-term scalability.

Methodology and Technical Approach

The development of the Peer Feedback App follows a structured, phased methodology beginning with frontend design and concluding with a fully integrated, secure backend system. During the Fall 2025 semester, the team focused on implementing the React frontend, developing the user interface components, routing structure, and feedback submission workflows. This phase emphasized wireframing, user-focused design, and building reusable UI components that will seamlessly integrate with the backend API.

The backend, planned for Spring 2026, will be developed using the Django framework. Django was selected for its robust security features, built-in administrative tools, and compatibility with SQL Server through the mssql-django database backend. The architecture includes a REST API built using Django REST Framework, responsible for handling authentication, assignment management, feedback submissions, and data retrieval for dashboard visualizations.

A critical component of the technical approach is maintaining student anonymity. To achieve this, the database schema will store user identities and feedback submissions in separate tables without a direct foreign key relationship. Instead, feedback entries will contain anonymous reviewer identifiers generated for each assignment. This ensures that even administrators with database access cannot match a specific comment to an individual student.

SQL Server serves as the system's central data store, hosting user records, courses, assignments, and aggregated analytics. All database communication will use encrypted connections, and role-based access controls will help restrict data visibility. OAuth 2.0 authentication, implemented through Django social authentication libraries, will provide secure login options using Google or university credentials.

Frontend and backend integration will rely on REST endpoints that deliver JSON responses. Tools such as Postman and Django's built-in API browser will support endpoint testing, while Pytest and Selenium will be used for backend and UI testing respectively. Development is managed through GitHub, with version control, branch protection, and issue tracking guiding the workflow.

This methodology ensures the system is built incrementally, tested thoroughly, and designed for long-term maintainability and security.

User Profiles/Personas

To ensure the design supports the needs and behaviors of all primary stakeholders, three representative user personas were developed: one for students, one for instructors, and one for administrators. These personas guide decision-making related to usability, accessibility, and workflows throughout the system.

Student Persona - Jordan Ruiz

Jordan Ruiz is a 20-year-old junior majoring in Computer Science. He frequently works in teams for project-based courses but often struggles to understand how his contributions are perceived by peers. Jordan wants clear, honest feedback that helps him improve but sometimes feels uncomfortable directly telling teammates where they need improvement. He uses his laptop daily for coursework and prefers tools with clean interfaces and simple navigation. For Jordan, anonymity is essential, as it helps him feel confident giving more candid and constructive evaluations. He values feedback that is structured and easy to understand, and he checks educational apps frequently through mobile devices.

Instructor Persona - Dr. Priya Menon

Dr. Menon teaches multiple sections of a software engineering course, each with several student teams. Tracking team performance and ensuring fair peer reviews across large groups is a consistent challenge. She needs tools that simplify the creation of assignments, ensure students participate on time, and provide intuitive dashboards that summarize group dynamics. Dr. Menon uses the university LMS extensively but finds its peer evaluation features limited. She values automated reminders, aggregated metrics, and data visualizations that allow her to quickly identify when intervention is needed.

Administrator Persona - Marcus Hale

Marcus is a systems administrator responsible for managing university academic applications. His priorities include maintaining secure authentication, ensuring compliance with university IT policies, and supporting faculty who use instructional tools. He needs a system with clear configuration settings, reliable user provisioning controls, stable performance, and strong data protection. Marcus rarely interacts with assignment-level features but is responsible for maintaining uptime and addressing security issues. He appreciates systems that integrate cleanly with existing SSO frameworks and require minimal maintenance.

These personas illustrate the varied goals and constraints across user groups and help shape design decisions throughout the development process.

Use Case Diagrams

Commented [EC4]: Add Content Here

The use case diagram for the Peer Feedback App reflects the core interactions between Students, Instructors, Administrators and the main functions required to support peer evaluation activities. While the frontend is currently implemented, several backend-driven use cases will be completed in Spring 2026.

Actors and Major Use Cases:

- Student:
  - Login, View Assignments, Submit Feedback, Complete Self-Assessment, View Dashboard, Review Past Feedback
- Instructor:
  - Create Assignments, Assign Review Groups, Monitor Submissions, View Analytics Dashboard
- Administrator:

- Manage Users, Configure System Settings, Maintain Data Compliance

A few use cases, such as authentication, data storage, and anonymity enforcement, are dependent on backend logic and will be available once the Django REST API is completed.

Use Case Diagram Description (Expanded Narrative)

The diagram highlights the system's core workflows and the responsibilities associated with each user role. Students primarily interact with the platform through feedback-related activities: logging in via OAuth 2.0, accessing assignment instructions, submitting structured and anonymous evaluations, and reviewing aggregated feedback through their personalized dashboard. The design ensures that no student ever sees identifiable information about their reviewers, supporting the system's commitment to anonymity.

Instructors play a more administrative role within the application. Their use cases allow them to create new peer review assignments, assign students to groups, and monitor participation. Through their instructor dashboard, they can view submission rates and summary analytics derived from student feedback. Although instructors can view aggregated trends, they cannot access individual reviewers' identities, preserving confidentiality.

Administrators oversee system-level functions such as user account configuration, auditing, and ensuring compliance with institutional policies. They do not interact with assignment-level workflows but play a key role in maintaining system security and performance. Their responsibilities include managing authentication frameworks and verifying that data storage complies with FERPA and other privacy guidelines.

The structured representation of these use cases provides a clear understanding of the system's functionality and the planned interactions that will be supported in the completed application.

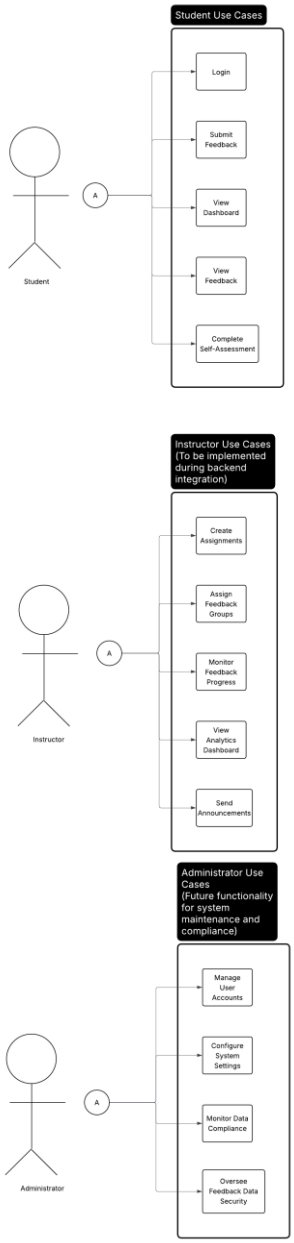

Figure 1. Use Case Diagram

Technical Architecture

The system architecture of the Peer Feedback App is designed to support a secure, scalable, and maintainable peer evaluation platform. The frontend, completed during the Fall 2025 semester, is developed using React and provides all user-facing pages required for assignment viewing, feedback submission, self-assessment, and dashboard interaction. React Router enables client-side navigation, while reusable components ensure a consistent and responsive interface across devices.

The backend, which will be implemented using Django, provides the core logic for authentication, data management, and secure handling of peer feedback. Django REST Framework will expose API endpoints that allow the frontend to interact with the backend through JSON-based requests. This separation of frontend and backend responsibilities supports modularity and makes the system easier to maintain and extend.

SQL Server serves as the main data store and is accessed via Django's mssql-django backend. The database schema includes tables for users, courses, assignments, feedback submissions, and analytics. To support the app's commitment to anonymity, the schema purposely avoids direct foreign key relationships between user accounts and specific feedback entries. Instead, anonymized reviewer IDs are generated for each assignment, ensuring that feedback cannot be traced back to individual students.

Authentication is handled through OAuth 2.0, allowing users to sign in using university credentials or Google SSO. Role-based access controls ensure that Students, Instructors, and Administrators each have access only to the features they are authorized to use. All communication between the frontend and backend will be encrypted, and data access patterns will follow the principle of least privilege.

The planned deployment environment includes Azure Web App Services, with GitHub Actions used for CI/CD pipelines once the backend is integrated. This architecture ensures that the application is secure, performant, and capable of supporting future enhancements as user needs evolve.

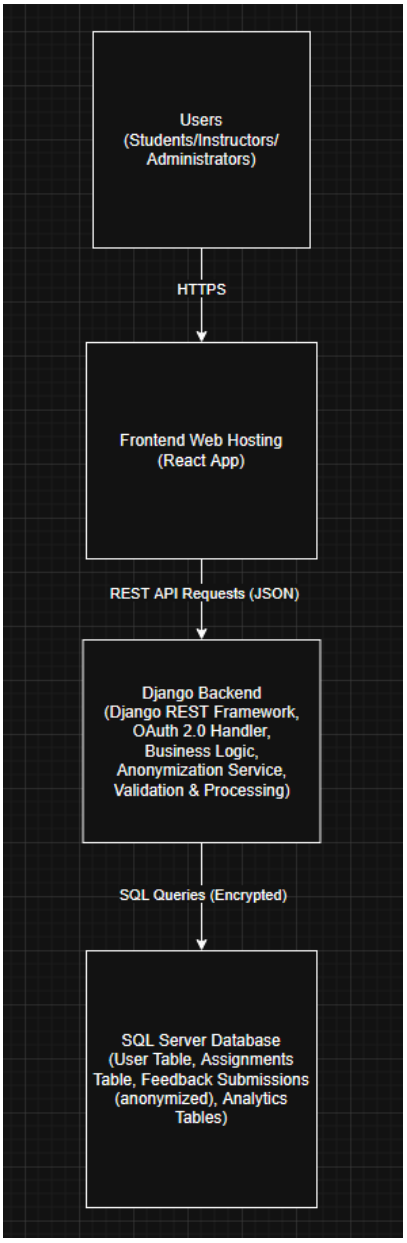

Figure 2. Logical Network Diagram

Testing

As of this stage in the project, testing has been primarily focused on the frontend implementation of the Peer Feedback App. Since backend development and full database integration are planned for the next phase, the current testing efforts are aimed to ensure the reliability, usability, and responsiveness of the user interface and its core interactive elements.

Testing Objectives

The main objectives of testing during the Fall semester were to:

- Verify that the React components render correctly and function as intended.
- Ensure smooth navigation between pages and responsiveness across devices and screen sizes.
- Validate user input handling and form behaviors in preparation for future backend connectivity.
- Identify and correct usability or accessibility issues before integration with the backend system.

Testing Methods

1. Component Testing:
    a. Individual React components, such as navigation menus, feedback forms, and dashboard placeholders, were tested in isolation using mock data. This ensured that each element displayed properly and responded to user actions without errors.
2. Interface and Layout Testing:

   a. Manual testing was performed to check layout consistency, alignment, and color contrast across browsers (Chrome, Firefox, Edge). Responsiveness was verified using browser developer tools to confirm that pages adapted correctly to both desktop and mobile viewports.

3. Form Validation Testing:
   a. Since feedback submission forms represent a major user interaction point, mock input validation was tested to ensure fields handled empty or incorrect input gracefully. Future integration will connect these validations to real-time server-side checks.
4. Navigation Flow Testing:
   a. Test users followed expected workflows, such as logging in through placeholder buttons, accessing the feedback page, and returning to the dashboard, to confirm intuitive page flow and prevent dead links or navigation errors.
5. Usability and Accessibility Review:
   a. Informal usability testing was conducted among team members and peers. Feedback emphasized clarity of interface elements and intuitive navigation. Adjustments were made to improve label visibility, button spacing, and readability. Basic accessibility checks were performed to meet WCAG 2.1 Level AA standards.

Testing Results and Findings

Testing confirmed that the current frontend prototype functions as intended within its scope. Pages render consistently across supported browsers, and the interface adapts effectively

to various screen sizes. Minor visual inconsistencies, such as spacing between form elements and inconsistent padding in dashboard cards, were identified and corrected.

While backend-dependent functionality (such as authentication and data persistence) remains untested at this stage, all interface components are ready for integration with the planned Django and SQL Server backend. Future testing efforts will expand to include API validation, database operations, and full user workflows once backend functionality is implemented.

Budget

Although the Peer Feedback App was developed using freely available software and university resources, a budget analysis was conducted to estimate the hypothetical cost of development in an industry setting. This analysis includes projected labor costs for each team member based on estimated hours worked, as well as software, hardware, and hosting expenses that would be required in a professional environment.

Budget Assumptions:

- Team of 4 developers (Ethan Cox, Advait Parab, Melanie Peck, Nathaniel Wisher)
- Estimated 150 hours per team member during the project period
- Average student developer wage: $25/hour (based on industry internship average)
- Use of free-tier tools and open-source technologies (React, Github, Visual Studio Code, etc.).
- Hypothetical costs for web hosting and domain registration

Estimated Cost Breakdown

Category	Description	Estimated Cost (USD)

Labor Costs	4 members x 150 hours x $25/hour	$15,000
Software & Tools	Free (React, Django, GitHub, Visual Studio Code)	$0
Cloud Hosting & Domain	Hypothetical one-year hosting plan and domain registration	$250
Testing Resources	Selenium	$0
Hardware Use	Personal computers and university lab access	$0
Miscellaneous	Documentation tools, graphic assets, reference materials	$0
Total Estimated Budget	----	$15,250

Figure 3. Project Budget Summary

Budget Discussion

In practice, no direct monetary expenses were incurred because the project utilized university infrastructure and open-source technologies. However, when considering the value of student labor, the estimated total project cost is approximately $15,250.

Labor represents over 98% of the total budget, highlighting that development time and expertise are the most significant resources in software projects. The use of open-source tools and university-provided infrastructure significantly reduced potential software and hosting costs.

As the project transitions in the next phase, integrating the backend and database, the team anticipates potential additional costs if professional cloud hosting, SSL certificates, or paid testing tools were required. These future costs are expected to remain modest compared to labor investments, with total expenditures likely not exceeding $17,000 under similar wage assumptions.

Project Plan

The development of the Peer Feedback App follows an agile, iterative project plan divided into multiple phases across the academic year. Each phase emphasizes incremental progress, testing, and refinement based on team feedback.

The Fall semester focused on frontend development, interface design, and initial usability testing. The Spring semester will concentrate on completing backend functionality, integrating authentication and database components, and finalizing system testing.

Development Phases

1. Planning and Requirements Gathering:
    a. Conducted through team discussions and literature review to identify user needs, ethical considerations, and core features of the application.
2. Frontend Design and Development:
    a. Implemented using ReactJS with a focus on user experience, accessibility, and responsive design. This phase established the framework for future backend integration.
3. Backend and Database Implementation (Upcoming):
    a. Planned for Spring 2026 using Django for server logic and SQL Server for secure data management. OAuth 2.0 will be used for authentication.
4. Testing and Integration:
    a. Includes integration of frontend and backend systems, validation of authentication flows, and performance testing under simulated user loads.
5. Final Review and Deployment:

a. The final phase will focus on user acceptance testing, documentation, and preparation for the final presentation.

Task #	*Task Name*	*Duration*	*Start Date*	*End Date*	*Purpose*
1	*Requirements Gathering*	*4 weeks*	*Mid-August*	*Mid-September*	*Collect and analyze requirements from potential users and stakeholders, focusing on core features and user needs.*
2	*UI/UX Design*	*4 weeks*	*Mid-September*	*Mid-October*	*Design wireframes and mockups for the app, emphasizing usability and accessibility for students.*
3	*UI/UX Testing*	*1 Week*	*Mid-October*	*Late October*	*Test mockup workflows and design*
4	*Frontend Development*	*5 weeks*	*Late October*	*Early January*	*Develop the app's user interface using React. Includes initial layouts, interactive elements, and form validation.*
5	*Frontend Testing*	*1 Week*	*Early January*	*Mid-January*	*Testing of the frontend of the site.*
6	*Backend Development*	*5 weeks*	*Mid-January*	*Late February*	*Set up the backend using Django. Focus on database integration, user authentication, and API creation.*

7	*Backend Testing*	*1 Week*	*Late February*	*Early March*	*Testing of Backend*
8	*Database Integration*	*3 weeks*	*Early March*	*Late March*	*Implement SQL for storing and managing user data, feedback submissions, and summaries.*
9	*Database Integration Testing*	*1 Week*	*Early March*	*Mid-March*	*Testing Database Integration*
10	*System Security and Authentication*	*2 Weeks*	*Mid-March*	*Late March*	*Integrate OAuth 2.0 for secure user authentication and ensure compliance with data privacy standards.*
11	*Security Testing*	*1 Week*	*Late March*	*Early May*	*Testing Security Features*
12	*Integration Testing*	*2 Weeks*	*Early May*	*Mid-May*	*Test the app for functional and non-functional requirements, including UI responsiveness and backend reliability.*
13	*Final Testing and Adjustment*	*2 Weeks*	*Mid-May*	*Early April*	*Final testing of project. Final minor adjustments (No functional changes unless required)*
14	*Presentation and Documentation*	*2 Weeks*	*Early April*	*April 14*	*Prepare project documentation and presentations*
		Extra Time left for buffer in case of falling behind Schedule			

Figure 4. Project Timeline

Project Plan Discussion

This plan allows the team to build iteratively while maintaining flexibility for adjustments based on feedback or unforeseen challenges. The frontend prototype completed in Fall 2025 will serve as the foundation for backend integration in Spring 2026.

Regular milestone reviews and weekly team meetings will ensure steady progress, while GitHub will continue to support task tracking, version control, and issue resolution. By the end of Spring 2026, the project is expected to achieve full functionality, with testing confirming that all core objectives: anonymity, usability, and structured feedback are successfully met.

Problems Encountered and Analysis of Problems Solved

Throughout the development of the Peer Feedback App, several challenges arose related to project coordination, frontend implementation, and technical constraints. While these issues occasionally slowed progress, each provided valuable learning experiences and informed improvements to the project workflow.

1. Responsive Design and Layout Inconsistencies

Problem:

Early builds revealed inconsistencies in page layouts between desktop and mobile displays. Certain components, like the feedback form and dashboard cards, did not scale properly on smaller screens.

Solution:

The team implemented responsive design principles using Flexbox and CSS Grid within the React components. Layouts were tested extensively using browser developer tools to

simulate multiple screen resolutions. These adjustments improved usability and ensured accessibility across devices.

2. Time Constraints and Task Prioritization

Problem:

Balancing course workloads and coordinating development schedules proved challenging, particularly during midterms. Some planned features, such as advanced form validation and improved animations, were delayed to meet presentation deadlines.

Solution:

The team reprioritized deliverables using a feature-based milestone approach, focusing on completing core pages before enhancements. This helped ensure that a fully functional prototype was available by the end of the semester, even if some features were deferred to Spring.

3. Limited Backend Connectivity

Problem:

Since backend development is scheduled for the next project phase, certain frontend components (such as feedback submission and user authentication) had to be simulated using placeholder data. This limited the ability to perform full end-to-end testing.

Solution:

To prepare for future integration, the team created mock API structures and standardized data models based on expected backend endpoints. This will streamline communication between the frontend and backend once integration begins.

Summary

Despite these challenges, the team successfully completed a functional, visually consistent frontend prototype. Each obstacle enhanced the group's collaboration, technical skills, and adaptability, laying the groundwork for a smoother backend integration process in the Spring semester.

Recommendations for Improvement

Based on the challenges encountered during the initial development phase, several recommendations have been identified to enhance both the technical implementation and team workflow for the next stage of the Peer Feedback App. These improvements will help streamline backend integration, strengthen security, and refine the user experience.

1. Implement Full Backend Integration Early

One of the most significant next steps is to establish the backend connection between the React frontend and the Django API. Early integration will allow for testing of real data interactions, authentication flows, and database operations well before the final testing phase. This will also make it possible to identify and resolve data-handling or performance issues sooner.

2. Introduce Continuous Integration and Deployment (CI/CD)

Setting up an automated CI/CD pipeline through GitHub Actions or Azure DevOps will improve consistency in builds and testing. Automating code linting, dependency checks, and deployment to a test environment would reduce manual setup time and ensure that new code changes are verified before merging.

3. Strengthen Testing Coverage

While the current phase emphasized manual and UI testing, future efforts should include automated integration tests and API endpoint validation once the backend is active. Incorporating unit testing within the frontend, using frameworks such as Jest or React Testing Library, will further improve reliability as the codebase expands.

4. Enhance Accessibility and Usability

Although basic accessibility checks have been conducted, further refinement is recommended. Conducting structured usability tests with a small group of target users (students and instructors) will provide insights into real-world navigation patterns, clarity of instructions, and overall satisfaction. Improvements may include better visual hierarchy, clearer labels, and accessibility features like keyboard navigation.

5. Improve Documentation and Knowledge Sharing

Expanding internal documentation will help maintain consistency as the project grows. A detailed developer guide outlining file structures, API endpoints, and component relationships should be created. Maintaining a project wiki or shared knowledge base on GitHub will also help future developers or maintainers understand the system architecture efficiently.

Summary

By focusing on early backend integration, stronger testing practices, and better documentation, the project can progress from a functional prototype to a robust, production-ready application. These recommendations not only address current limitations but also position the Peer Feedback App for scalability, security, and long-term sustainability in academic environments.

CONCLUSION

Lessons Learned

The development of the Peer Feedback App provided the team with valuable experience in applying classroom knowledge to a real-world software project. Although the current phase focused primarily on frontend development, the project's scope required continuous learning, collaboration, and problem-solving that extended beyond technical implementation.

One of the most significant lessons learned was the importance of clear communication and project coordination. Early challenges with version control and workflow consistency underscored the need for structured collaboration. Establishing standardized Git practices and regular progress meetings proved essential in maintaining momentum and reducing errors.

Another key takeaway involved the balance between design goals and time management. The team learned to prioritize core functionality over aesthetic enhancements when faced with limited time. This helped ensure that the application's essential features: navigation, feedback forms, and dashboards were functional and tested by the semester's end.

From a technical standpoint, the team gained a deeper understanding of React development, responsive design, and component-based architecture. Working with dynamic UI elements provided insight into building scalable, modular interfaces that can later integrate with backend systems.

Equally important were lessons in user experience and accessibility. Through iterative testing and peer feedback, the team recognized the importance of designing with empathy, ensuring that the app is inclusive, easy to navigate, and visually clear for all users.

Finally, the project highlighted the value of ethical and secure design practices. Given the sensitivity of peer feedback data, the team studied data privacy principles such as FERPA and

GDPR compliance, reinforcing the need to build trust and transparency into every layer of the application.

Overall, the lessons learned from this phase of the project have prepared the team for more advanced development in the upcoming semester. The combination of technical proficiency, collaboration, and ethical awareness gained through this process will directly inform the successful completion of the Peer Feedback App.

Abilities and Skills Developed Throughout Project

The Peer Feedback App project provided the team with an opportunity to apply academic concepts to a practical development scenario, strengthening both technical and professional abilities. The collaborative nature of the project encouraged individual growth in specialized areas while reinforcing the importance of teamwork in software development.

Technical Skills Developed

Frontend Development (ReactJS):

Team members gained hands-on experience building a dynamic and modular interface using React. This included working with reusable components, managing state efficiently, and ensuring responsive layouts for various screen sizes.

Version Control and Collaboration (Git/GitHub):

The project required consistent collaboration through GitHub, where the team practiced branching strategies, code reviews, and conflict resolution. These practices enhanced coordination and improved understanding of professional version control workflows.

UI/UX Design Principles:

Through iterative design and testing, the team improved its understanding of usability, accessibility, and interface hierarchy. Members learned to apply user-centered design principles and incorporate feedback from informal usability tests.

Testing and Debugging:

Although testing focused mainly on the frontend, the team became familiar with Selenium for automated UI testing and manual methods for validating layout, form functionality, and navigation flow. This experience built a foundation for future integration and system testing.

Project Management and Agile Development:

Working under an agile-inspired structure helped the team develop planning and prioritization skills. Regular sprint-style updates, task assignments, and milestone tracking prepared the group for real-world software development cycles.

Professional and Interpersonal Skills Developed

Communication and Team Coordination:

Regular meetings and use of shared project management tools improved the team's ability to communicate clearly, delegate responsibilities, and resolve conflicts efficiently.

Problem-Solving and Adaptability:

The process of resolving issues, ranging from merge conflicts to layout inconsistencies, taught members to diagnose problems methodically and adapt to evolving requirements.

Ethical Awareness and Data Privacy Considerations:

Researching and planning for compliance with FERPA and GDPR increased awareness of the ethical responsibilities that come with developing software involving personal data. This strengthened the team's understanding of security, consent, and transparency in software systems.

Plans For Spring

The upcoming Spring semester will focus on transforming the Peer Feedback App from a functioning frontend prototype into a complete, fully integrated web application. The team's primary goal will be to connect the user interface to a secure and efficient backend, enabling live data handling, authentication, and feedback storage.

Backend Development and Integration

The first priority will be to implement the Django backend and integrate it with the SQL Server database. This phase will include developing RESTful API endpoints for user registration, feedback submission, and dashboard data retrieval. The integration will enable real-time interaction between the user interface and persistent data storage, completing the system's core functionality.

Authentication and Data Privacy

Following backend setup, the team will implement OAuth 2.0 authentication, allowing users to log in using university or external credentials. This phase will also focus on ensuring compliance with FERPA and GDPR requirements. Particular attention will be given to protecting user anonymity and securing all data transmissions through encryption and token-based access control.

System Testing and Optimization

Once integration is complete, comprehensive testing will be conducted. This will include integration testing, API validation, and user acceptance testing (UAT) to ensure that the app functions as intended across all components. Performance testing will verify that the system remains stable and responsive under typical student usage loads. Testing feedback will guide final optimizations and bug fixes.

Deployment and Presentation

The final weeks of the semester will focus on preparing the application for presentation at the IT Expo. The team will finalize documentation, create a technical poster, and prepare a live demonstration for the IT Expo. The deployment will showcase the app's key capabilities, anonymous peer feedback, self-assessment tracking, and intuitive usability.

REFERENCES AND APPENDIX

References

Association for Computing Machinery. (2018). *ACM code of ethics and professional conduct*. ACM. https://www.acm.org/code-of-ethics

Ciscell, G., Foley, L., Luther, K., Howe, R., Gjsedal, T. (2016). Barriers to Accessing Tutoring Services Among Students Who Received a Mid-Semester Warning. *Learning Assistance Review, 21(2).* https://files.eric.ed.gov/fulltext/EJ1114513.pdf

Erasmus University Rotterdam. (n.d.). *Peer feedback*. Impact At The Core | Erasmus University Rotterdam. https://www.eur.nl/en/impactatthecore/knowledge-platform-impact-driven-education/designing-impact-driven-education/feedback-assessment/peer-feedback

European Union. (2016). *General Data Protection Regulation (GDPR), Regulation (EU) 2016/679*. Official Journal of the European Union. https://gdpr-info.eu

Family Educational Rights and Privacy Act of 1974, 20 U.S.C. § 1232g. (1974). https://www.govinfo.gov/content/pkg/USCODE-2011-title20/html/USCODE-2011-title20-chap31-subchapIII-part4-sec1232g.htm

Peer feedback. Impact At The Core | Erasmus University Rotterdam. (n.d.). https://www.eur.nl/en/impactatthecore/knowledge-platform-impact-driven-education/designing-impact-driven-education/feedback-assessment/peer-feedback

Simonsmeier, B. A., Peiffer, H., Flaig, M., & Schneider, M. (2020). Peer feedback improves students' academic self-concept in higher education. *Research in Higher Education*, *61*(6), 706–724. https://doi.org/10.1007/s11162-020-09591-y

State of California. (2018). *California Consumer Privacy Act of 2018, Cal. Civ. Code §§ 1798.100–1798.199*. https://oag.ca.gov/privacy/ccpa

United States. (1990). *Americans with Disabilities Act of 1990, 42 U.S.C. § 12101 et seq.* https://www.ada.gov/resources/ada-law-and-regulations/

United States. (1998). *Children's Online Privacy Protection Act of 1998, 15 U.S.C. §§ 6501–6506*. https://www.ftc.gov/legal-library/browse/rules/childrens-online-privacy-protection-rule-coppa

Appendix

Commented [EC5]: Add Content Here

Appendix A – Frontend Interface Mockups

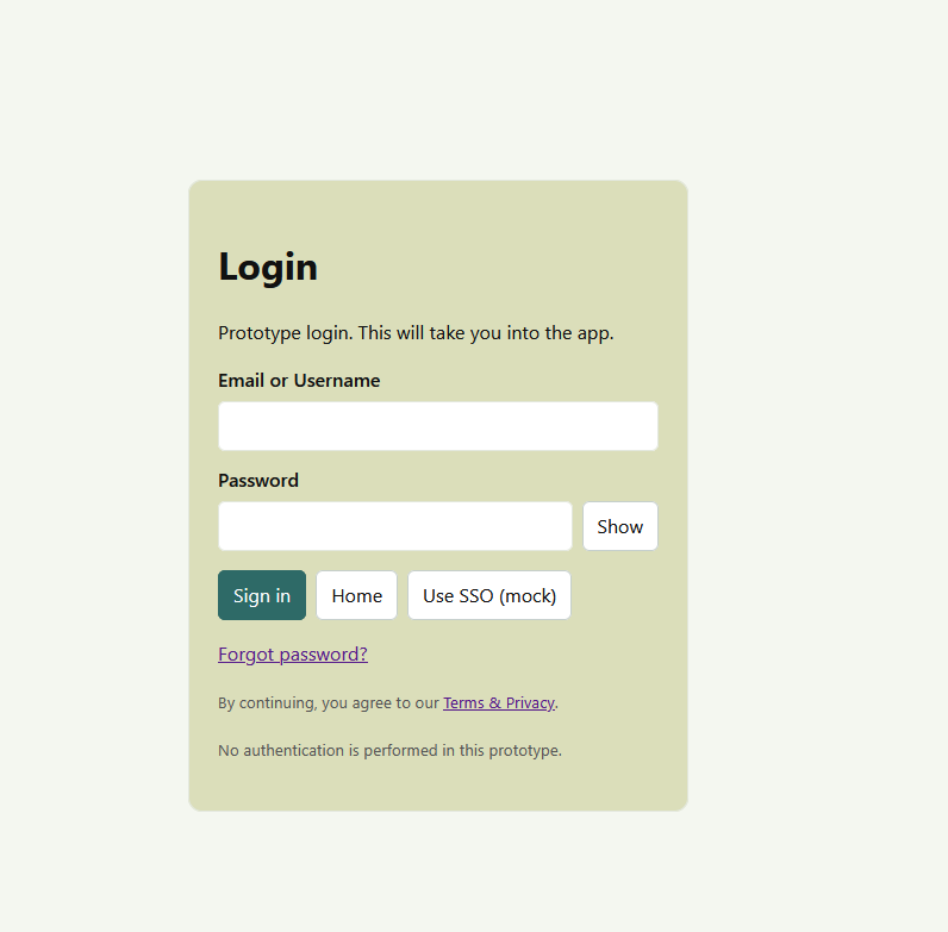

Figure 5. Login Page

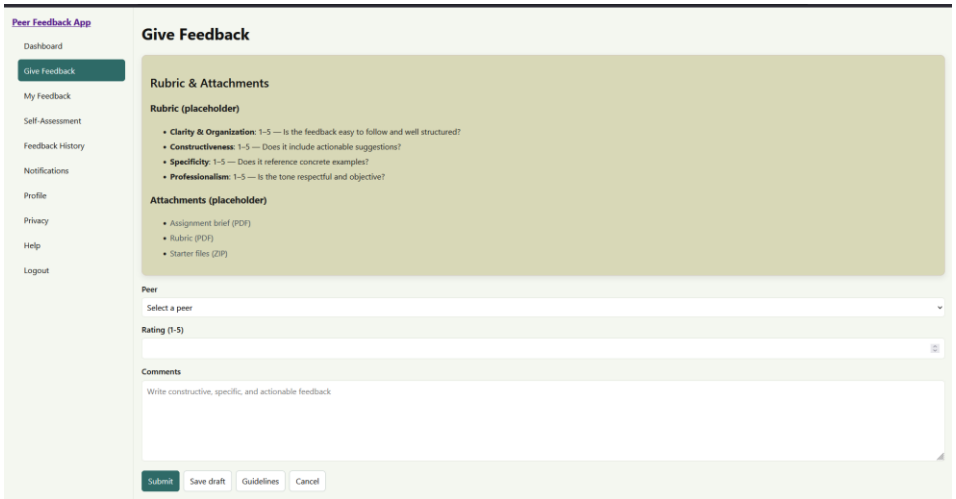

Figure 6. Feedback Submission Form

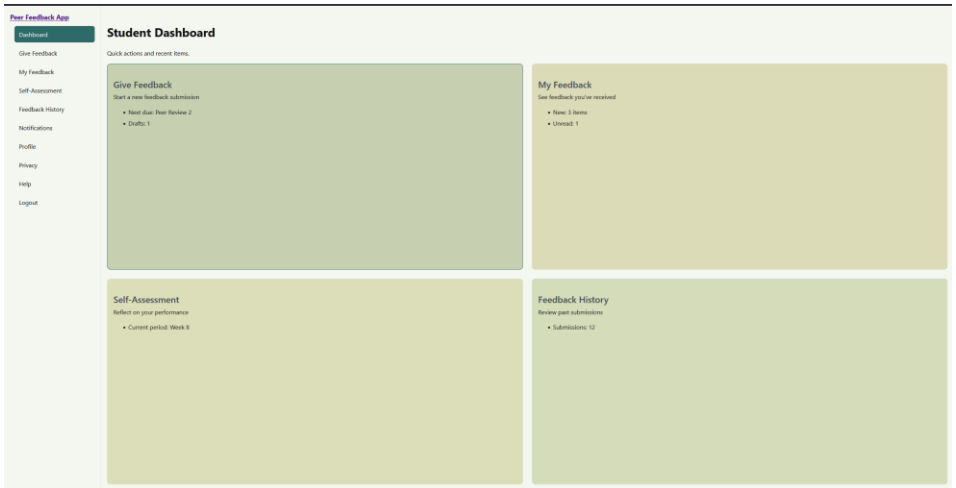

Figure 7. Student Dashboard

Appendix B – Technologies and Tools

Category	Technology / Tool	Purpose / Use Case
Frontend	React	Build user interface with modular components
Backend	Django	Server-side logic and API endpoints

Database	SQL Server	Store feedback, user data, and analytics
Authentication	OAuth 2.0	Secure login with Google or university accounts
Version Control	Git, Github	Source management and collaboration
Testing	Selenium, Pytest	UI automation testing. Backend unit testing
Design	Figma, Canva	UI mockups and visual prototyping

Figure 8. Table of Technologies and Tools